WESTERN GOVERNORS UNIVERSITY • COURSE D278

Scripting & Programming Foundations

The Complete Exam Cheat Sheet — All 11 Chapters

1 · Introduction | 2 · Variables | 3 · Branches | 4 · Loops | 5 · Arrays | 6 · Functions | 7 · Algorithms | 8 · Design / SDLC | 9 · Languages & Libraries | 10 · Troubleshooting | 11 · Debugging

Course competencies: (1) identify scripts for program requirements · (2) use fundamental programming elements · (3) explain the logic & outcome of simple algorithms.
Code examples use **Coral**, the pedagogical language/flowchart tool used throughout the course. Assessment = 1 final exam covering 3 competency units.

HOW TO READ THIS GUIDE

Each chapter gives the definitions, the exact **Coral** syntax, worked patterns, reference tables, and diagrams you need. **Key Point** boxes = must-know facts. **Watch Out** boxes = the exact traps exams test. **Tip** boxes = memory hooks. Nothing here assumes prior programming knowledge — every term is defined on first use.

Program = Input → Process → Output (the mental model behind everything)

A **computer program** is a sequence of instructions a computer carries out, one at a time, in order. A useful analogy: a program is like a **recipe** — an ordered list of steps. Almost every program follows three stages: **1 Input** get data (from a user, file, or sensor) → **2 Process** compute/decide → **3 Output** show a result.

Flowchart shapes (Coral draws these automatically):

Shape	Meaning
Oval	Start / End (terminator)
Parallelogram	Input or Output
Rectangle	Process / assignment
Diamond	Decision (branch)

1 Introduction to Programming

1.1–1.2 Programs & programming basics

- A **program** automates a task by executing statements top-to-bottom.
- Output with `Put x to output`; read input with `x = Get next input`.
- The computer executes statements **in order**; changing the order changes the result.

1.3 & 1.9 Comments and whitespace

- A **comment** is text for humans, ignored by the computer. In Coral use `// like this`.
- **Whitespace** (spaces, blank lines, indentation) is **mostly ignored** by the computer but is vital for human **readability**.

1.4 Brief history & 1.5 Computers all around us

- Computing moved from mechanical devices to electronic computers; the **information age** is roughly a century old and ongoing.
- **Embedded computers** live inside other devices (cars, appliances, thermostats) — most computers are embedded, not desktops.
- Desktop use is flat/declining while **mobile/smartphone** use keeps rising.

KEY POINT — EVERYTHING IS BITS

Inside a computer, **all** data (numbers, characters, images, sound) is stored as **bits**: 0s and 1s. A single 0/1 is a **bit**; 8 bits = 1 **byte**.

1.6 Representing information as bits

- **Binary** is base-2. Ex: binary `1100` = decimal **12**.
- **ASCII** encodes characters in **7 bits** → 128 characters (letters, digits, punctuation).
- **Unicode** (1991) uses more bits to represent 100,000+ characters, including non-English symbols/emoji.
- A word/string is stored as a **sequence of bit codes**, one per character.

1.7–1.8 Problem solving & why programming

- Programming is mostly **problem solving**: creating a methodical, step-by-step solution to a task.
- **Computational thinking** = precise, logical thinking; programming rewards **attention to detail** (one wrong symbol can break a program).
- Computing careers are numerous, well-paid, and growing; programming also benefits non-computing jobs.

1.10 Code vs pseudocode

- **Pseudocode** = an informal, human-readable sketch of a program (not runnable).
- Coral offers two synced views of the same program: a **flowchart** and **code**.

CHAPTER 1 RECAP

Computers surround us and are growing · all data is bits (0/1) · programming = methodical problem solving · attention to detail matters · pseudocode is an informal description of a program.

2 Variables & Assignments

2.1–2.2 Variables & assignment

- A **variable** is a named location in memory that holds one value.
- An **assignment** statement stores a value into a variable using `=`.
- Assignment works **right-to-left**: the right side is computed, then stored in the left variable.

```
integer numPeople
numPeople = 5       // store 5
numPeople = numPeople + 3  // now 8
```

WATCH OUT — = IS NOT ==

`=` means **assign** (put a value in a variable). `==` means **compare** for equality (Ch 3). `x = x + 1` is legal (increment), not algebra.

2.3 Identifiers (naming rules)

- An **identifier** is a name for a variable/function.
- May contain **letters, digits, underscores**; **cannot start with a digit**.
- **Case-sensitive** (`numCars` ≠ `numcars`); reserved words can't be used.
- Convention: **camelCase** (`userAge`); use descriptive names.

2.4–2.5 Arithmetic expressions

Op	Meaning	Example
`+`	add	`a + b`
`-`	subtract	`a - b`
`*`	multiply	`a * b`
`/`	divide	`a / b`
`%`	modulo (remainder)	`a % b`

Precedence: `( )` first, then `* / %`, then `+ -` (left to right).

2.7 Floating-point numbers

- A **float** holds numbers with a fractional part (e.g., `3.14`, `2.5`).
- An **integer** holds whole numbers only.

2.8 Using math functions

Built-in functions take **arguments** in `( )` and evaluate to a value.

```
SquareRoot(49.0)        // 7.0
RaiseToPower(2.0, 3.0)  // 8.0
AbsoluteValue(-4)       // 4
```

2.9 Random numbers

- `RandomNumber()` returns a pseudo-random integer.
- `SeedRandomNumbers()` seeds the generator; the next value is computed from the previous one (deterministic given a seed).

WATCH OUT — 2.10 INTEGER DIVISION

If **both** operands of `/` are integers, division **truncates** (drops the fraction): `10 / 4` → **2** (not 2.5); `3 / 4` → **0**. Make one operand a float to get floating-point division.

2.11 Type conversions & casts

- A **type conversion** changes one type to another (e.g., integer → float).
- A **type cast** explicitly converts a value's type so the right operation happens (e.g., cast an int to float before dividing).

2.12 Modulo operator

- `%` gives the **remainder**: `17 % 5` → **2**.
- Common uses: even/odd (`x % 2 == 0`), extracting digits, wrap-around.

WATCH OUT — DIVIDE BY ZERO

The second operand of `/` or `%` must never be **0** — a divide-by-zero error occurs at **runtime** and terminates the program.

2.13 Common data types

Type	Holds	Example
`integer`	whole numbers	`42`
`float`	decimals	`3.14`
`character`	one symbol	`'A'`
`string`	text	`"hello"`
`boolean`	true / false	`true`

2.14 Constants

A **constant** is a named value that **cannot change** after it is set. Use constants to name fixed values (rates, limits, physical constants) so code is readable and easy to update. Convention: **UPPER_CASE** names.

```
constant float SOUND_SPEED = 761.207
// use SOUND_SPEED instead of a "magic number"
```

CHAPTER 2 RECAP

Variable = named storage · `=` assigns right-to-left · identifiers can't start with a digit & are case-sensitive · int/int truncates · `%` = remainder · never divide by 0 · casts convert types · constants hold unchangeable values.

3 Branches (Decisions)

IF · ELSE · LOGIC

3.1–3.2 Branches: if / if-else / if-elseif-else

A **branch** is a sequence of statements executed only when a condition is true. In a flowchart a **decision** (diamond) creates a **true** branch and a **false** branch.

```
// if — do something only when true
if userAge > 65
   hotelRate = hotelRate - 20

// if-else — choose one of two paths
if score >= 60
   Put "Pass" to output
else
   Put "Fail" to output

// if-elseif-else — choose one of many
if temp > 90
   Put "Hot" to output
elseif temp > 60
   Put "Warm" to output
else
   Put "Cold" to output
```

KEY POINT — PICK THE RIGHT STRUCTURE

Use **if** for one optional action · **if-else** for two mutually-exclusive paths · **if-elseif-else** when exactly one of several cases applies · **multiple separate ifs** when conditions are independent and more than one can run.

3.4 Detecting ranges

Combine relational operators with **and** to test a range:

```
if (nthPerson >= 1) and (nthPerson <= 5)
   Put "First five" to output
```

3.3 Equality & relational operators

Operator	Meaning
==	equal to (equality)
!=	not equal to
< / >	less than / greater than
<= / >=	less-or-equal / greater-or-equal

Two-character operators (>=, <=, ==, !=) must be written exactly — you can't rearrange the symbols.

3.5 Logical operators: and, or, not

A **logical operator** treats operands as true/false and evaluates to true/false.

Op	True when…
and	both sides are true
or	at least one side is true
not	flips true ↔ false

3.6 Order of evaluation (precedence)

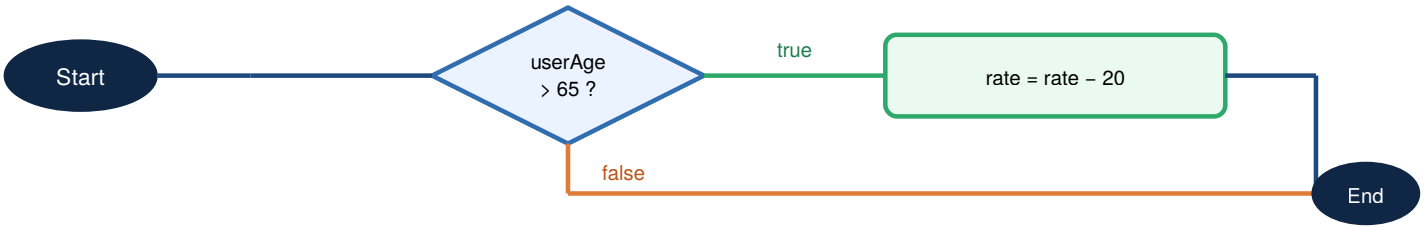
Operators evaluate in this order (highest first):

( ) · not · * / % · + − · < <= > >= · == != · and · or

Use parentheses to make intent explicit and avoid mistakes.

WATCH OUT — 3.8 FLOATING-POINT COMPARISON

Never test floats with `==`: tiny rounding errors make equal-looking values differ. Instead check that the difference is within a small range: `AbsoluteValue(a - b) < 0.0001`.



3.1 A decision creates a true branch and a false branch; both paths rejoin afterward.

CHAPTER 3 RECAP

Branch = conditional statements · `==` tests equality (not `=`) · relational: `< > <= >= == !=` · logical: `and or not` · precedence: () → not → arithmetic → relational → equality → and → or · compare floats with a tolerance, never `==`.

4 Loops (Repetition)

4.1–4.2 What a loop is

A **loop** repeatedly executes its body while a condition is true. Each pass is an **iteration**. The condition is checked, the body runs, and control returns to the condition.

4.6 while loop

Checks the condition **first**; runs the body only while it's true (may run **0** times).

```
integer curPower
curPower = 2
userNum = Get next input
while userNum == 1
   Put curPower to output
   Put "\n" to output
   curPower = curPower * 2
   userNum = Get next input
Put "Done." to output
```

4.4–4.5 & 4.10 for loop (loop N times)

A **for** loop packs three parts on one line: **init ; condition ; update**. Ideal for counting.

```
integer i
integer sum
sum = 0
for i = 0; i < 3; i = i + 1
   sum = sum + i
Put sum to output   // 0+1+2 = 3
```

KEY POINT — WHILE VS FOR

Use a **for** loop when you know/count the number of iterations (loop **N** times). Use a **while** loop when you repeat until a condition changes and the count isn't known in advance (e.g., "keep reading until the user enters −1").

4.9 do-while loop

Executes the body **first**, then checks the condition — so it always runs **at least once**.

```
do
   userNum = Get next input
while userNum != 0
```

4.7 Nested loops

A **nested loop** is a loop inside another loop (an **inner** loop inside an **outer** loop). For each single pass of the outer loop, the inner loop runs fully. Used for grids, tables, and pattern printing.

```
for r = 0; r < 3; r = r + 1
   for c = 0; c < 3; c = c + 1
      Put "*" to output
   Put "\n" to output
```

WATCH OUT — INFINITE LOOPS

If the loop condition **never becomes false**, the loop runs forever. Always change a variable in the body that moves the condition toward false (increment a counter, read new input, etc.).

4.3 Common loop patterns (memorize these)

Sum

```
for i=0; i<n; i=i+1
   sum = sum + val
```

Count matches

```
if val < 0
   negCount = negCount + 1
```

Find maximum

```
if val > maxVal
   maxVal = val
```

CHAPTER 4 RECAP

while = check then run (0+ times) · **for** = counted iterations (init;cond;update) · **do-while** = run then check (1+ times) · nested loop = loop within a loop · always progress toward the exit to avoid an infinite loop.

5 Arrays

LISTS · INDICES · ITERATION

5.1 Array concept

- An **array** is an ordered list of items of the **same data type and fixed size**.
- Each item is an **element**; its position is an **index**.
- A single-value (non-array) variable is a **scalar** variable.
- Analogy: a scalar is a **truck** (one container); an array is a **train** (many cars).

KEY POINT — INDICES START AT 0

A 5-element array has valid indices **0,1,2,3,4**. The last index is **size − 1**. Element `x[0]` is the first item, not `x[1]`.

5.2 Declaring & accessing arrays

```
integer array(5) itemCounts   // 5 elements
itemCounts[0] = Get next input
itemCounts[1] = 99
Put itemCounts[1] to output   // 99
```

`arrayName.size` gives the number of elements — use it to loop safely.

5.3–5.4 Iterating & finding max/min

Walk every element with a **for** loop from `0` to `size − 1`.

```
integer i
integer maxVal
maxVal = userVals[0]        // seed with 1st element
for i = 0; i < userVals.size; i = i + 1
   if userVals[i] > maxVal
      maxVal = userVals[i]
```

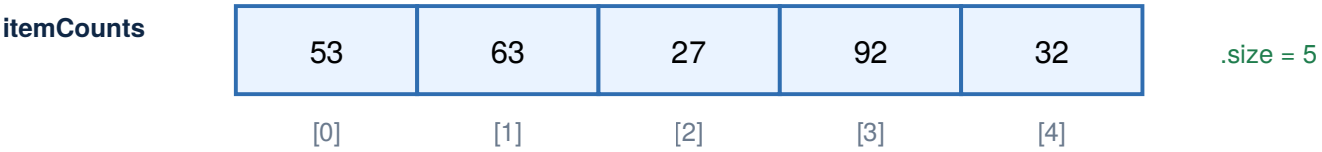
WATCH OUT — SEEDING MAX/MIN

Initialize `maxVal` to the array's **first element**, not to 0. Seeding with 0 fails if every value is negative (0 would wrongly "win").

5.5 Swapping two variables

To swap two values you need a **third, temporary** variable (like using a free hand to swap two objects).

```
integer temp
temp = x        // save x
x = y           // x gets y
y = temp        // y gets old x
```



5.1 An array stores many same-type elements, each reached by a 0-based index.

CHAPTER 5 RECAP

Array = fixed-size ordered list of one type · indices are 0-based, last = size−1 · iterate with a for loop to `.size` · seed max/min with element [0] · swapping needs a temp variable · scalar = single-value variable.

6 User-Defined Functions

PARAMETERS · RETURN · MODULARITY

6.1 Function basics

- A **function** is a named group of statements you run by **calling** its name.
- A **parameter** is an input named in the function **definition**.
- An **argument** is the actual value passed in the function **call**.
- A function's **local variables** exist only inside it (**scope**).

```
// definition: PrintPizzaArea has 1 parameter
Function PrintPizzaArea(float pizzaDiameter)
         returns nothing
   float piVal
   float pizzaRadius
   piVal = 3.14159265
   pizzaRadius = pizzaDiameter / 2.0
   Put piVal * pizzaRadius * pizzaRadius to output

// call: 12.0 is the argument
PrintPizzaArea(12.0)
```

KEY POINT — PARAMETER VS ARGUMENT

Parameter = the variable in the definition (a placeholder). **Argument** = the real value you supply when calling. The argument is copied into the parameter.

6.2 Return values

A function can **return** a value with a **return statement**, which sends the value back and exits the function. If it returns nothing, use `returns nothing`.

```
Function ComputeSquare(integer x)
         returns integer y
   y = x * x
   return y

// 4 + 36 = 40
sumOfSquares = ComputeSquare(2)
             + ComputeSquare(6)
```

A function call is an **expression** — it evaluates to its returned value and can be used inside larger expressions.

6.3 Why define functions?

- **Readability** — Main stays short and understandable.
- **Reuse** — write once, call many times.
- **Modularity** — build/test pieces separately, then combine.
- Guideline: a function usually shouldn't exceed ~20 lines.

6.3 Modular & incremental development

- **Modular development**: divide a program into separate functions that can be developed/tested independently.
- **Incremental development**: write & test a **small** amount of code, then add and test more.

6.6 Array parameters

```
Function SumList(integer array(?) numList)
         returns integer total
   for i = 0; i < numList.size; i = i + 1
      total = total + numList[i]
   return total
```

`array(?)` means the function accepts an array of any size; use `.size` inside.

CHAPTER 6 RECAP

Function = reusable named block · **parameter** (definition) vs **argument** (call) · `return` sends a value back & exits · functions improve readability/reuse/modularity · build incrementally · pass arrays with `array(?)` .

7 Algorithms

CORRECTNESS · EFFICIENCY · SEARCH

7.1 What an algorithm is

- An **algorithm** is a sequence of steps that solves a problem, producing correct output for **any valid input**.
- **Correctness** is the top priority — an algorithm must work for all valid inputs, not just one example.
- Before coding, a programmer first designs a correct algorithm.

7.2 Computational problems

A **computational problem** is a problem solvable by a computer. It specifies exactly three things:

Part	Meaning	Example
Input	the data given	an array of integers
Question	what's asked of the input	what is the maximum value?
Output	the desired result	the maximum integer

- An **algorithm** describes the steps that solve a computational problem.
- One problem can be solved by **many** algorithms — finding the best one is the work.
- Computational problems recur across domains (e-commerce, biology, transportation), so recognizing that a task *is* a known problem lets you reuse a known algorithm.

7.3 Algorithm efficiency

- Efficiency = how much **runtime** (operations) and **memory** an algorithm uses.
- **Best case**: fewest operations; **worst case**: most operations. Analyze the worst case to be safe.
- An algorithm with **polynomial** runtime is considered **efficient**; exponential is not.
- Keep the input as a variable so runtime can be described as it grows.

7.4 Linear search

Check each element from the start until the key is found. Works on **any** list (sorted or not).

```
Function LinearSearch(integer array(?) numbers,
                      integer key)
         returns integer keyIndex
   keyIndex = -1
   for i = 0;
       (i < numbers.size) and (keyIndex == -1);
       i = i + 1
      if numbers[i] == key
         keyIndex = i
```

Worst case: key is last or absent → checks all N elements.

7.5 Binary search

Faster search that requires a **sorted, directly-accessible** list (like an array). Repeatedly check the **middle** and discard half the remaining elements.

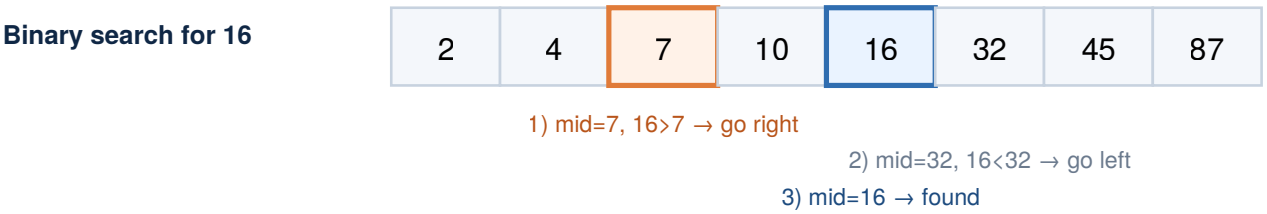
```
Function BinarySearch(integer array(?) numbers,
                      integer key)
         returns integer mid
   integer low
   integer high
   low = 0
   high = numbers.size - 1
   while (high >= low) and (numbers[mid] != key)
      mid = (high + low) / 2
      if numbers[mid] < key
         low = mid + 1
      elseif numbers[mid] > key
         high = mid - 1
```

KEY POINT — LINEAR VS BINARY

Linear: works on any list, up to **N** checks. **Binary**: needs a **sorted** array, about **$\log_2(N)$** checks — e.g., 1,024 items → ~10 checks, not 1,024. Binary search halves the problem each step.

7.6–7.7 Sorting & the knapsack problem

- **Sorting** arranges elements in order (e.g., by repeatedly **swapping** out-of-order pairs).
- **0-1 Knapsack**: choose items to maximize value within a weight limit. Trying all combinations is optimal but slow; a **heuristic** makes a fast, "good-enough" (possibly non-optimal) choice.



7.5 Each comparison discards half the list, so binary search finds the key in ~$\log_2(N)$ steps.

CHAPTER 7 RECAP

Algorithm = correct steps for any valid input · efficiency = runtime + memory (worst case matters; polynomial = efficient) · linear search = any list, ≤N checks · binary search = sorted array, ~$\log_2$N checks · heuristics trade optimality for speed.

8 The Design Process (SDLC, Objects, UML)

SOFTWARE DESIGN

8.1 System Development Life Cycle (SDLC)

The **SDLC** is the set of phases used to build software:

Phase	Purpose
Analysis	Define the program's **goals** (what to build)
Design	Define the specifics of **how** to build it
Implementation	**Write** the program (code)
Testing	Check the program correctly **meets the goals**

8.2 Objects

- An **object** groups related **data (variables)** and **operations (functions)** into one higher-level unit.
- Objects model real things (a Car object bundles its data and behaviors), keeping large programs organized.

8.5 Waterfall vs Agile

Waterfall	Agile
Phases done once, in sequence	Short repeated cycles (iterations)
Each phase can be long	Implementation repeats many times
Hard to change late	Adapts to change continuously

8.3–8.4 UML diagrams

UML (Unified Modeling Language) is a standard set of diagrams for modeling software.

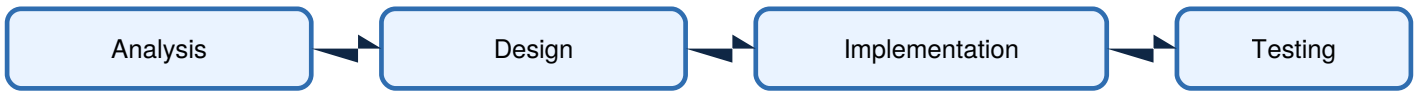
Diagram	Shows	SDLC phase
Use case	What the user can do / goals	Analysis
Class	Classes: names, data members, functions	Design
Activity	Flow of actions (like a flowchart)	Implementation
Sequence	Order of events/messages over time	Testing

KEY POINT — DIAGRAM ↔ PHASE

Use case → goals (analysis) · **Class** → structure/how (design) · **Activity** → program steps (implementation) · **Sequence** → event order for verifying I/O (testing).

WATCH OUT — STRUCTURAL VS BEHAVIORAL

A **class diagram is structural** (it shows structure, and can show data-member types). A **sequence diagram is behavioral** and shows the order of events — it does **not** contain program code.



8.1 The four SDLC phases (waterfall runs them once in order; agile repeats them in cycles).

CHAPTER 8 RECAP

SDLC = Analysis → Design → Implementation → Testing · object = data + operations bundled · UML diagrams: use case / class / activity / sequence · waterfall = once-through, agile = iterative.

9 Software Languages & Libraries

9.1 Compiled vs interpreted

	Compiled	Interpreted (scripting)
How it runs	A **compiler** converts the whole program to machine code first	An **interpreter** runs it one statement at a time
Speed	Runs **faster**	Runs **slower**
Portability	Machine-specific build	Runs on any machine with the interpreter
Examples	C, C++, Java*	Python, JavaScript, MATLAB

A scripting language is another name for an interpreted language.

9.1 Static vs dynamic typing

- **Statically typed**: a variable's type is fixed and checked before running (e.g., C, C++, Java).
- **Dynamically typed**: a variable's type can change while running (e.g., Python). Ex: `x = "Hi"` then `x = 5` is allowed.

9.1 Other language kinds

- **Object-oriented** languages have strong support for objects (e.g., C++, Java, Python).
- **Markup languages** describe the structure/format of a document rather than compute — e.g., **HTML** for web pages (tags like `<title>`, *italic*, **bold**).
- Kinds are **not exclusive** — a language can be object-oriented *and* dynamically typed.

9.2 Libraries

- A **library** is a set of **pre-written functions** for common tasks (math, statistics, graphics).
- Libraries **improve productivity**: reuse tested code instead of writing everything from scratch.
- A program can include several libraries for different purposes.

KEY POINT — QUICK CLASSIFIERS

Faster + machine code = compiled · **runs line-by-line, portable** = interpreted · **type fixed** = static · **type can change at runtime** = dynamic · **HTML** = markup · **pre-written functions** = library.

CHAPTER 9 RECAP

Compiled (C/C++/Java, fast) vs interpreted (Python/JS/MATLAB, portable) · static vs dynamic typing · object-oriented & markup (HTML) languages · libraries = reusable pre-written functions that boost productivity.

10.1 Troubleshooting process

- **Troubleshooting** is a **systematic** process for finding and fixing a problem's cause.
- A **hypothesis** is a proposed cause. A **test** is a procedure whose result **validates** or **invalidates** that hypothesis.
- The core loop: form a hypothesis → run a test → validate or invalidate → repeat until the cause is found.

KEY POINT — TEST OUTCOMES

A good test should clearly **validate** (confirm) or **invalidate** (rule out) a hypothesis. If a test can't do either, it isn't useful.

10.2 Asymmetric tests

An **asymmetric test** gives a more conclusive answer on one outcome than the other. Ex: plugging a lamp into a known-working outlet — if it works, "lamp is broken" is invalidated (conclusive); if it doesn't, the result is less certain.

10.3 & 10.6 Creating hypotheses from knowledge

- Start with the **most-likely** and **most easily-testable** hypotheses first.
- Hypotheses and tests come from **knowledge** of how the system works — the more you learn, the better your hypotheses.
- Create hypotheses in **groups**, then test them in a sensible order.

10.10 Hierarchical hypotheses

Hypotheses can form a **tree** traversed top-down: a broad hypothesis splits into sub-hypotheses until a specific cause is pinpointed.

10.x Binary search when troubleshooting

To locate a fault in a large system, split it in **half**, test which half contains the problem, discard the good half, and repeat — quickly narrowing down the cause (same halving idea as binary search).

10.1 Troubleshooting is a cycle: a test either validates a hypothesis (cause found) or invalidates it (try the next).

CHAPTER 10 RECAP

Troubleshooting = systematic hypothesis→test→(in)validate cycle · asymmetric test = more conclusive one way · test most-likely/easiest first · knowledge drives hypotheses · hierarchical hypotheses = a tree · halve the system to localize faults.

11.1 Basic debugging

- **Debugging** = finding and fixing **bugs** (errors) in a program. It applies the troubleshooting cycle (hypothesis → test → validate/ invalidate) to code.
- Even one wrong character among thousands can cause a bug, so debugging is a normal, essential skill.

11.1 Debug output statements

Insert temporary **output statements** that print variable values so you can see what the program is actually doing and compare it to what you expect.

```
celsiusValue = 100
Put "DEBUG: celsiusValue is " to output
Put celsiusValue to output
fahrenheitValue = celsiusValue * (9.0 / 5.0) + 32
```

WATCH OUT — CLEAN UP AFTER

Remove (or disable) debug output statements **once the cause is found**, so they don't clutter real output.

11.2–11.5 Common bug categories

Bug type	Symptom / cause
Calculation error	Wrong arithmetic/formula (e.g., int division, wrong operator)
Logic error	Wrong condition/branch → wrong result for some inputs
Loop error	Off-by-one, wrong bounds, or never-ending loop
Function error	Wrong parameters/return, bad call, or scope mistake

11.x Hierarchical / binary-search debugging

- Split the code into **regions** and test which region contains the bug, then split that region again — a **binary search** through the code to localize the fault fast.
- **Hierarchical debugging**: form broad hypotheses, then narrow to specific lines (a tree of hypotheses, like Ch 10).

KEY POINT — A TEST EITHER CONFIRMS OR RULES OUT

Debugging mirrors troubleshooting: run the program, form a hypothesis about the faulty statement, add a debug test to **validate** or **invalidate** it, and repeat until the bug is isolated and fixed.

11.6 Programming knowledge

Beyond basic debugging, more advanced techniques exist (assertions, debugging tools, unit tests), but the hypothesis-driven approach above is the universal foundation.

CHAPTER 11 RECAP

Debugging = troubleshooting for code · debug output statements reveal actual variable values · bug types: calculation, logic, loop, function · localize bugs by halving the code (binary-search debugging) · remove debug prints when done.

★ Coral Quick-Reference Card

SYNTAX AT A GLANCE

Declarations, I/O & assignment

```
integer count
float price
constant float PI = 3.14159
count = Get next input
count = count + 1
Put "Total: " to output
Put count to output
Put "\n" to output   // newline
```

Branch

```
if x > 10
   Put "big" to output
elseif x == 10
   Put "ten" to output
else
   Put "small" to output
```

Function

```
Function Cube(integer n) returns integer c
   c = n * n * n
   return c
```

Loops

```
// while
while n > 0
   n = n - 1

// for (count N times)
for i = 0; i < n; i = i + 1
   Put i to output

// do-while (runs ≥ 1x)
do
   x = Get next input
while x != 0
```

Arrays

```
integer array(5) vals
vals[0] = 10
Put vals.size to output   // 5
for i = 0; i < vals.size; i = i + 1
   Put vals[i] to output
```

Operators (high → low precedence)

`( )` | `not` | `* / %` | `+ −` | `< <= > >=` | `== !=` | `and` | `or`

`/` truncates for int÷int · `%` = remainder · compare floats with a tolerance, never `==`.

EXAM-DAY ONE-LINERS (ALL 11 CHAPTERS)

Ch1 data = bits; ASCII 7-bit/128 chars. **Ch2** int÷int truncates; never ÷0; `%` =remainder; constants don't change. **Ch3** `==` compares, `and/or/not` ; float compare = tolerance. **Ch4** while(0+)/for(counted)/do-while(1+); avoid infinite loops. **Ch5** arrays 0-indexed, last=size−1; swap needs temp. **Ch6** parameter(def) vs argument(call); `return` exits. **Ch7** linear=any list ≤N; binary=sorted ~$\log_2 N$. **Ch8** SDLC = Analysis→Design→Implementation→Testing; UML use-case/class/activity/sequence; waterfall vs agile. **Ch9** compiled(fast) vs interpreted(portable); static vs dynamic typing; HTML=markup; library=pre-written functions. **Ch10** hypothesis+test→validate/invalidate; asymmetric & hierarchical hypotheses. **Ch11** debug with output statements; bug types calc/logic/loop/function; binary-search the code.